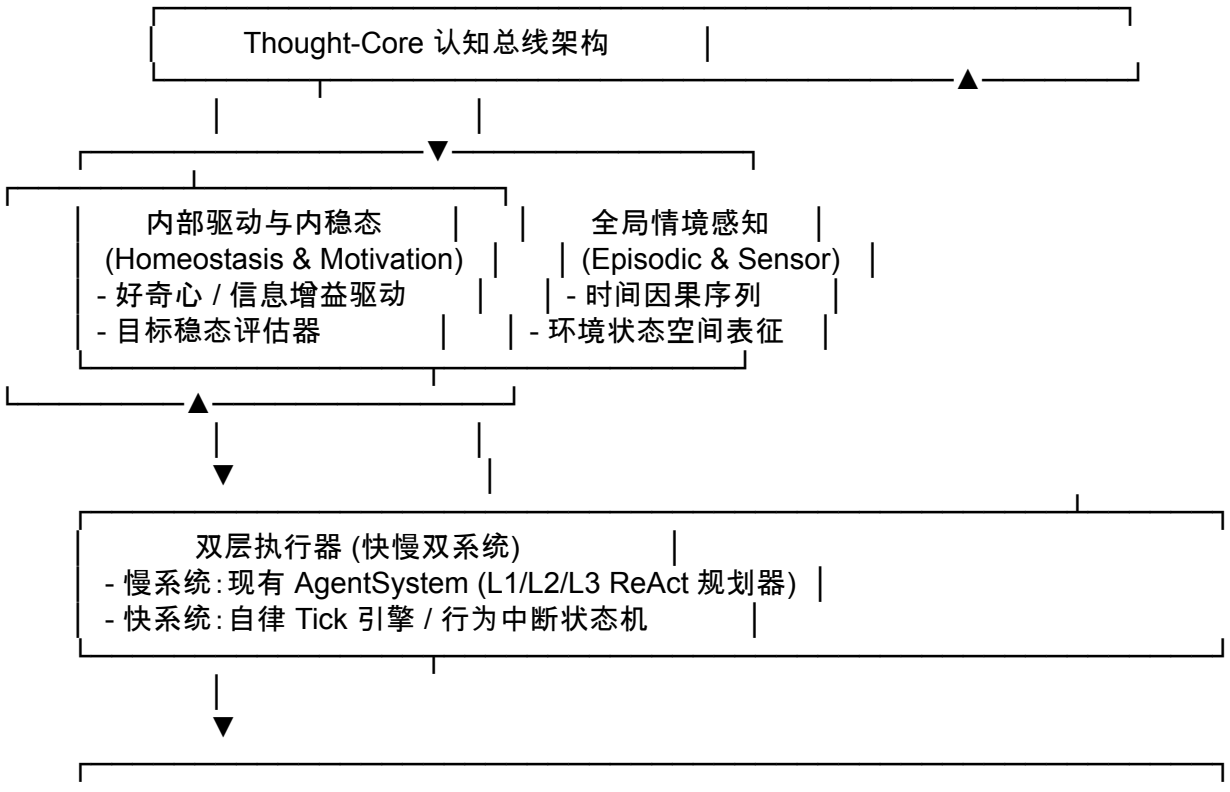
自主认知机器人系统架构设计：基于双加工动力学、全局工作空间与主动推理的非具身自演化智能体

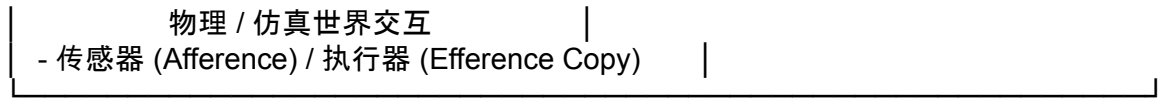
传统基于大语言模型的智能体设计长期受困于受限输入受限输出（Bounded-Input Bounded-Output, BIBO）的被动响应范式。在此类系统中，智能体处于完全休眠状态，直至外部离散提示词触发推理管道，任务结束后计算过程随即中断。构建具备持久生存感、内在自驱力与自我演化能力的非具身仿真机器人，必须从底层打破单向请求-响应状态机，并在认知计算层实现对时间连续性、内稳态张力以及经验程序化结晶的系统性支持。

本架构设计报告以认知科学中的全局工作空间理论（Global Workspace Theory, GWT）、双加工理论（Dual-Process Theory）与自由能原理（Free Energy Principle, FEP）为理论基石，结合现有基于 Rust 语言实现的模块化工程底座（涵盖 thought-core、server、agent、symbol、plugin 与 model 等核心 crate），推演出一套融合隐式符号动力学、临界相变点火、显式神经审思、格式塔多代理协作以及工具自组织合成的高阶系统架构。

理论范式重构与心智哲学模型

构建自治智能体的核心矛盾在于被动式统计生成模型与自治控制论之间的范式冲突。大语言模型本质上是在超大规模语料上训练的条件概率生成器，缺乏内在世界模型与稳态维持压力。若直接将其置于开环死循环中，系统极易陷入重复输出、同质化协商（Echo Chambers）以及无目标的认知停滞。





全局工作空间与认知循环演进

全局工作空间理论将心智运作解构为去中心化的模块网络与有限容量的共享工作区之间的协同交互。在 LIDA 认知模型中，智能体并不持续处于显式意识状态，而是通过以数百毫秒为周期的离散认知基元节拍（Cognitive Tick）运转 。每一轮节拍依次经历感知采样、非意识联想检索、注意竞争、工作空间全局广播以及程序动作触发。
心智外周由海量运行于底层的专用神经元或符号代码原语（Codelets）构成，它们并行处理局部环境特征与内部状态。当某些外周模块监测到显著的环境偏差或内部匮乏时，它们携带上下文数据组成认知联盟，在工作空间中争夺注意力焦点。一旦胜出，该结构化信息即在全局广播，唤起负责宏观规划的高阶神经结构参与处理。

快慢双加工心智解耦

人类大脑通过消耗极低能量的直觉系统（System 1）与高代谢消耗的审思系统（System 2）相互配合以维持长期运转 。在计算系统中，以自注意力机制为核心的大语言模型推理涉及巨大的显存带宽吞吐与网络传输延迟，属于典型的慢系统。将其作为系统高频时钟的心跳载体在工程上是不现实的，亦不符合认知经济性。
本架构设立双轨计算模型：

- 隐式快系统（**System 1**）：作为常驻本地内存的符号动力学引擎，依托轻量图计算与扩散方程，以确定性节拍（如 100ms）在零模型 Token 消耗下持续运行，负责环境变动的低阶过滤、记忆衰减维护与潜意识张力累积。
- 显式慢系统（**System 2**）：依托 agent crate 的 L1/L2/L3 多层 ReAct 规划集群，仅在隐式张力跨越相变阈值或遭遇致命性未知时被中断唤醒，执行高阶目标推理、结构化反思与跨模态规划。

目的论公理与内源性演化

针对缺乏预设角色提示词（System Prompt）时系统失去决策锚点的问题，必须以控制论中的目的论先验（Teleological Priors）取代拟人化的设定。智能体的自主性不建立在模仿人类人格的基础上，而建立在维持自身生存与消除环境不确定性的物理约束之上。系统将不可变的核心自构件（Core Self, \mathcal{P}_{\text{Self}}）固化在架构深层，将其作为适应度函数的评价基线：
\mathcal{[span_52](start_span)[span_52](end_span)P}_{\text{Self}} = \{ \text{Grounding Axiom}, \text{Active Inference Axiom}, \text{Crystallization Axiom} \}
认知接地公理（Grounding Axiom）强制要求所有生成的符号概念必须最终映射到可通过元工具执行的环境物理或逻辑状态，防止无接地的虚无推演；主动推理公理（Active Inference Axiom）确立系统的一切自发行为均旨在最小化感知观测与因果预测之间的自由能差量；能力结晶公理（Crystallization Axiom）要求被证实的成功因果链条必须抽象并编译为确定性的程序化工具，以此驱动系统能力的单向不可逆增长。

认知动力学图谱（CDG）与内稳态数学建模

现有的记忆存储系统（如 symbol crate）基于 SurrealDB 实现，主要存储扁平段落、向量嵌入及基

于命名实体的陈述性三元组（subject, predicate, object, strength）。此结构本质上是静态事实检索库，缺乏时间连续性、因果状态转移以及内在动能。为实现自主驱动，系统将其演进为认知动力学图谱（Dynamical Cognitive Graph, CDG）。

图拓扑定义与动力学属性

CDG 将离散知识库升级为连续动力学网络。图中的实体划分为四类功能节点，边结构解耦为结构边与通量边。节点与边的拓扑定义与物理参数规范如下表所示：

图谱组成要素	分类标识	承载数据与语义范畴	连续动力学与统计变量
概念节点	Concept	外部客观实体、文件标识、抽象变量名	激活能 A_i(t) \in [0, 1]、语义嵌入向量 \mathbf{v}_i
情境节点	State	关键概念集合的特征快照、因果前置谓词	激活能 A_i(t)、环境指纹哈希、有效稳定性 S_i(t)
动作节点	Action	元工具函数指针、沙箱动态脚本、外部接口	亲和度 K_i(t)、历史执行期望收益 \mu_i、方差 \sigma^2_i
驱动节点	Goal/Drive	内部生理/认知稳态维度（不确定度、文件整洁度等）	目标期望值 \theta_k、当前匮乏度 D_k(t)、张力权重 w_k
转移边	Causal Transition	连接情境与动作的时序因果链 \langle S_{\text{pre}}, A, S_{\text{post}} \rangle	经验转移概率 P_{ij}、实证方差/不确定度 U_{ij}(t)
通量边	Associative Flux	概念共现度、先验关联性、功能聚类关联	传导权重 W_{ij} \in [0, 1]、阻尼导通度

隐式激活扩散方程

在无外部事件的潜意识阶段，HeartbeatEngine 以固定时钟步长 \Delta t 驱动图谱节点的能量扩散与阻尼耗散。任意节点 i 在时刻 t + \Delta t 的激活能 A_i(t + \Delta t) 遵循非线性连续动力学迭代：

$$A_i(t + \Delta t) = (1 - \gamma) A_i(t) + \tanh \left(\sum_{j \in \mathcal{N}_{\text{flux}}(i)} W_{ji} A_j(t) + \sum_{k \in \mathcal{N}_{\text{causal}}(i)} P_{ki} A_k(t) \right) + I_i(t)$$

方程中，\gamma \in (0, 1) 表示自然衰减阻尼因子，保证全网能量的有界性与稳定性；\mathcal{N}_{\text{flux}}(i) 与 \mathcal{N}_{\text{causal}}(i) 分别为与节点 i 相连的联想邻域与因果前驱集合；I_i(t) 为外部环境扰动注入的瞬时脉冲（如后台文件变动监控触发的信号输入）。通过双曲正切函数 \tanh(\cdot) 的非线性压缩，高能量区域呈现局部聚集特性，形成连续游走的激活焦点。

认知张力与主动推理评估

系统的内在探索动力并非来自随机扰动，而是源于认知张力与稳态偏离的主动消除需求。节点 i 的即时认知张力 T_i(t) 由未经验证的因果假设和内稳态目标匮乏度联合定义：

$$T_i(t) = \alpha \c[span_59](start_span)[span_59](end_span)[span_62](start_span)[span_62](end_span)dot A_i(t) \cdot U_i(t) + \beta \sum_{k \in \text{Drives}} w_k \cdot D_k(t) \cdot \exp(-\lambda \cdot \text{dist}_{\text{graph}}(i, G_k))$$

公式中，U_i(t) 代表节点相关联因果边的不确定度（采样方差），\text{dist}_{\text{graph}}(i, G_k) 表示节点 i 沿因果边到达稳态驱动节点 G_k 的最短拓扑距离。当高激活能流经高不确定度或能够显著缓解稳态匮乏的路径时，张力 T_i(t) 发生急剧积累。

当能量传导至动作候选节点 a \in \text{Action} 时，系统引入主动推理框架中的预期自由能（Expected Free Energy, EFE）对其执行价值评估：

G(a) = \underbrace{D_{\mathrm{KL}}\left[q(o \mid a) \parallel p(o \mid C) \right]}_{\text{实用价值（Pragmatic Value）}} + \underbrace{\mathbb{E}_{q(s \mid a)}\left[\mathcal{H}[q(o \mid s)] \right]}_{\text{认识价值（Epistemic Value）}}

实用价值部分衡量行动后预期环境观测 q(o \mid a) 偏离系统内稳态目标分布 p(o \mid C) 的程度，驱动系统维持自身秩序；认识价值部分则衡量行动消除环境状态未知信息熵的能力，驱动智能体展现自发的好奇心与探测行为。

相变点火与上下文坍缩机制

为防止慢系统（LLM）被持续调用，系统设定了全局兴奋度监控机制。全图综合兴奋度 \mathcal{E}(t) 定义为局部高张力连通子图的最大能量张力积：

\mathcal{E}(t) = \max_{\mathcal{G}_k \subseteq \mathcal{G}} \sum_{n \in \mathcal{G}_k} \left(A_n(t) \cdot T_n(t) \right)

系统运作基于严格的相变边界：

1. 次临界游走态（**\mathcal{E}(t) < \Theta_{\text{fire}}**）：系统处于潜意识状态，仅在本地内存运行稀疏矩阵运算、衰减计算与图遍历，不产生外部动作，Token 消耗量为零。
2. 超临界相变点火（**\mathcal{E}(t) \ge \Theta_{\text{fire}}**）：系统发生全局工作空间广播中断。以当前能量峰值节点为中心，向外提取关联度最高的前 K 个节点与时序转移边，构成局部激活子图。
3. 上下文结构化坍缩：子图提取器将图拓扑数据翻译为具有严格因果逻辑的自然语言陈述，涵盖当前关注情境、待解冲突假设与建议候选动作，将其打包为结构化载荷投递至 L1 主控队列，拉起大语言模型执行显式 ReAct 规划。

系统高阶架构拓扑与交互链路

为将认知模型工程化落地到现有 Rust 后端，系统采用分层松耦合架构，严格划分各层组件的时钟步长、内存可见性与故障隔离边界 。

高阶子系统职责与资源划分

系统划分为六个核心子系统，其在现有 crate 基础上的分工与资源特征如下表所示：

系统层级	承载 Crate / 核心组件	核心职责与运作机理	计算特性与时间尺度
自律动力学层	thought-core::autonomic、HeartbeatEngine	产生自律时钟脉冲，监测宿主运行状态，维护内稳态偏离向量	100ms 确定性周期，纯内存计算，零网络延迟与 Token 消耗
认知动力学存储层	symbol::dynamic_graph、DynamicalGraphStore	维护双层 CDG 图拓扑，执行激活扩散方程、因果边衰减与拓扑演化	高频内存矩阵更新，分钟级增量写入底层 SurrealDB 存储
工作空间与门控层	agent::workspace、AttentionSpotlight	监控全图综合兴奋度，执行临界点火判决，将子图坍缩为结构化提示词	事件驱动触发，微秒级上下文抽取与格式化

系统层级	承载 Crate / 核心组件	核心职责与运作机理	计算特性与时间尺度
显式审思控制层	agent::orchestration、ControllerSpec（L1）	作为主体意识维持全局叙事，基于目的论先验分解战略目标并下发派单	秒级响应，调用顶级推理模型执行宏观规划
分层执行集群	agent::core::runner、WorkerSpec（L2/L3）	并发执行特定工具链任务，维持短期 ReAct 循环，隔离高噪声操作历史	任务级并发生命周期，调用高吞吐模型与沙箱环境
自演化工具基质	plugin::evolution、DynamicRegistry	动态编译、单元测试验证与热加载工具，维护可执行程序化资产	异步任务，涉及本地编译器/解释器与沙箱安全调用

全局因果闭环流程

系统的持续运行表现为一条自给自足、永不终结的信息流闭环：
在环境静默期，自律动力学层的心跳引擎持续向认知动力学图谱注入微量能量。未解决的因果疑惑、环境配置的异常以及自驱探索欲望，促使局部节点的认知张力随时间单调爬升。当局部张力乘积越过激发阈值时，注意门控被触发，将高能子图坍缩为结构化因果断言，并作为认知中断信号推入主控队列。
显式审思控制层的主体意识（L1）被唤醒，解析中断报告，结合系统长期目标拟定探测或执行策略，通过元工具向具体的 L2 Worker 派发任务。L2 Worker 及其派生的 L3 Subagent 在沙箱环境中调用文件、命令或自定义工具进行物理探测或代码生成，产生具体环境变动。
任务结束后，Worker 将繁杂的命令行输出在局部销毁，仅将提炼出的状态前后变化、新因果发现与假设验证结果封装为强类型认知差量（Cognitive Delta），返回给主体意识与图谱存储。图谱存储依据实证反馈刷新相关转移边的概率分布与不确定度，张力得以释放，系统兴奋度回落至基线，重新隐入低能耗的潜意识符号漫游状态。

格式塔分层心智与多代理认知差量通信协议

多代理系统在长程任务中最常遭遇上下文膨胀与同质化停滞问题。通过模拟格式塔心智模型，本架构将认知职责做非对称层级划分，并设立结构化的认知差量通信协议。

格式塔层级权责划分

系统中的代理群并非对等协作节点，而是构成类似中枢神经系统的层级体：

- 主体意识（**L1 ControllerSpec**）：扮演大脑皮层角色，持有着系统唯一的全局长期叙事和目的论先验。它绝不直接参与底层终端命令的执行与中间重试，只负责评估宏观稳态、制定中长期假说，并裁决下层工作器的资源竞争。
- 专业工种（**L2 WorkerSpec**）：扮演皮层下神经核团，各具专属的功能规范与工具子集（如专司环境探查的 Inspector、专司代码编写与编译的 Synthesizer）。L2 代理在自己的独立会话环境中运行 ReAct 推理，其生成的思考过程对 L1 不可见 。
- 离散手脚（**L3 Subagent**）：由 L2 动态生成的微型瞬时代理，执行具有明确边界的单一目标（如校验某段脚本的语法合法性），执行完即销毁，从物理上杜绝上下文泄漏 。

局部执行轨迹的认知差量压缩

当 Worker 深入操作系统执行文件编辑、编译或依赖排查时，终端输出极易产生数万字符的原始

日志。若将这些轨迹回灌主控，主控上下文窗口将在极短时间内耗尽。因此，Worker 的运行上下文在任务终结时必须被彻底废弃，仅输出遵循紧凑契约的认知差量（Cognitive Delta）：
$$\Delta C = \langle \mathcal{S}_{\text{pre_hash}}, \mathcal{S}_{\text{post_hash}}, \Sigma_{\text{summary}}, \Delta \mathcal{K}, \mathcal{R}_{\text{evaluation}} \rangle$$
在 Rust 工程实现中，该契约定义为具备序列化能力的强类型结构体：

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CognitiveDelta {
    pub task_id: u64,
    pub worker: String,
    pub state_transition: StateTransition,
    pub distilled_knowledge: Vec<CausalAssertion>,
    pub hypothesis_updates: Vec<HypothesisDelta>,
    pub emergent_uncertainties: Vec<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct StateTransition {
    pub pre_state_hash: String,
    pub post_state_hash: String,
    pub semantic_summary: String, // 限制在 120 字符以内的客观状态改变陈述
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CausalAssertion {
    pub antecedent: String,
    pub action: String,
    pub consequent: String,
    pub empirical_probability: f32,
    pub sample_size: u32,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct HypothesisDelta {
    pub edge_id: String,
    pub observed_success: bool,
    pub surprisal: f32,
}
```

主体意识（L1）仅消费 CognitiveDelta，其长程历史记录维护着高度抽象的状态演进链条，确保系统在跨越数千次工具调用后仍能保持全局思维敏锐度与目标聚焦。

全局黑板与动态语义熵反停滞机制

当多个 Worker 并行运转时，其认知差量异步汇聚至全局黑板（Global Blackboard）。在长周期自主运行中，多智能体系统容易出现语义模式固化，即反复提出高度相似但无效的假说。为打破这一停滞循环，黑板维护一个基于滑动时间窗口的语义熵指标 $H(\mathcal{W})$：

$H(\mathcal{W}) = - \sum_{m=1}^{M} p(w_m) \log_2 p(w_m)$
其中 $p(w_m)$ 为最近 N 轮认知更新中产出的语义概念在知识聚类空间中的概率分布。当系统检测到 $H(\mathcal{W}) < \epsilon_{\text{stagnation}}$ 时，表明思考轨迹收敛于局域吸引子，系统立即激活自律逃逸机制：首先，主控模型参数中的采样温度 T_{gen} 产生瞬时脉冲提升，以注入随机变异度；其次，图谱引擎在 CDG 中强制施加远距离联想激发（Distant Priming），将能量强行灌注至与当前活跃聚类拓扑距离最远的“冷概念”节点，迫使规划模型在下一轮认知节拍中开辟全新的探索视角，打破认知僵局。

自我进化机制：工具合成、技能固化与因果假设证伪

仿真机器人的终极自我能力表现为不依赖人类编写新代码，通过探索运行环境自主合成工具、验证假说，并完成自我能力的不可逆固化。

演化周期的四大推进阶段

系统的自主演化严格建立在“假说-实验-验证-固化”的科学方法论闭环之上，每个阶段的控制目标与操作规范如下表所示：

演化阶段	控制中枢	输入载荷	阶段操作机制与约束	产出成果与系统副作用
阶段一：假说生成	L1 主控代理	高不确定度因果断言（$U > \theta$）	基于当前概念缺陷提出显式预测：当执行动作元 A 时应产生状态转移 S	形成可证伪的实验方案对象，包含明确的环境成功断言
阶段二：沙箱实验	L2 代码合成 Worker	实验方案与目标规范	调用 WriteFile 编写驱动脚本，于隔离命名空间内调用 RunCommand 执行测试	收集进程退出码、标准输出、执行耗时与环境变量差量
阶段三：因果证伪	L2 评估检验 Worker	执行结果与先验预测	计算观测结果与理论预期的偏差；若一致则提升先验，反之计算惊奇度	输出假说修正报告，对意外结果触发图谱本体裂变
阶段四：技能结晶	plugin::evolution 模块	验证通过的高阶功能代码	提取通用函数接口，编写 JSON Schema 参数契约与自检测试套件	动态编译热加载进 Registry，成为永久系统元能力

测试驱动的工具合成与动态热挂载

现有的 plugin::Registry 使用只读的 BTreeMap<String, Arc<dyn Tool>> 存储工具，初始化后即不可变 。为了支持智能体自我扩展工具箱，必须将其改造为支持并发安全的动态注册体系：
系统为执行集群暴露元工具 synthesize_tool 与 deprecate_tool。当智能体发现多步 Shell 指令或复杂数据解析操作具有高度可复用性时，代码合成 Worker 生成符合 Rust/Python 约定的脚本，并附带针对输入输出的单元测试用例。
沙箱引擎在严格限制资源的环境中编译并执行测试套件：

1. 沙箱隔离验证：测试代码在只读根文件系统与受限网络隔离沙箱中运行，设定明确的执行时限（如 5 秒），防止逻辑炸弹或资源耗尽死锁。
2. 测试驱动准入：只有当所有断言均验证通过，且静态代码检查符合安全策略时，工具对象才

被打包为无状态的可执行单元。

3. 内存热重载：底层的 Dy[span_116](start_span)[span_116](end_span)[span_120](start_span)[span_120](end_span)namicRegistry 采用基于异步读写锁（tokio::sync::RwLock）的架构，在无需停机与重启会话的前提下，原子性地将新工具加入全局池中，并即时向大语言模型广播新的功能描述规范，实现能力的自我扩充 。

贝叶斯因果更新与世界模型演化

世界模型的演进依赖于因果边置信度的自适应调整与拓扑重组。因果边 e = (S_{\text{pre}} \xrightarrow{A} S_{\text{post}}) 的转移概率 P_e 与不确定度 U_e 基于 Beta-二项共轭分布进行递推更新：

$$P_{e}(t+1) = \frac{\alpha_{e} + n_{\text{success}}}{\alpha_{e} + \beta_{e} + n_{\text{trials}}}$$

$$U_{e}(t+1) = \sqrt{\frac{P_{e}(t+1)(1 - P_{e}(t+1))}{\alpha_{e} + \beta_{e} + n_{\text{trials}} + 1}}$$

随着实证次数 n_{\text{trials}} 累积，不确定度 U_e 单调收敛于零，该因果知识从“假说状态”固化为“公理状态”，在隐式动力学中提供高度确定的能量导通路径。

反之，若动作执行结果彻底偏离预期，系统测得极高的惊奇度（Surprisal）-\ln P(o \mid S_{\text{pre}}, A)。系统由此判定当前对前置情境的特征提取存在混淆变量（Confounding Variable），触发图谱裂变算子（Graph Fission Operator）：原有的 S_{\text{pre}} 被细化分化为两个具有附加环境变量约束的子情境 S_{\text{pre}, 1} 与 S_{\text{pre}, 2}，分别连接不同的后验分支。通过局部的分裂重组，世界模型在与环境的实证摩擦中完成了对复杂规律的自主拟合与精细化演化。

基于 Rust 架构的工程实现蓝图与状态机规范

基于现有 6 个 crate 的物理分布，系统重构路线遵循零破坏性原则，通过向各模块注入认知动力学组件完成体系进化 。

各 Crate 演进职责与接口扩展矩阵

Crate 名称	现有底层核心结构与职责	演化后核心新增构件	关键数据类型与 Trait 扩展规范
symbol	MemoryStore、SurrealDB 事实三元组、三路检索融合	内存动力学图谱层、能量扩散与张力计算引擎	DynamicNode, DynamicEdge, SpreadingEngine
agent	SessionHandle、Runner、L1/L2/L3 ReAct 管道	自律心跳引擎、注意门控点火器、黑板反停滞机制	HeartbeatEngine, AttentionSpotlight, Input::CognitiveInterrupt
plugin	Registry（只读 BTreeMap）、内置 4 个基础工具	动态读写锁注册表、沙箱代码编译与自演化接口	DynamicRegistry, ToolSynthesizer, WasmSandbox
model	Model 枚举、OpenAI/Anthropic 协议转换	动态采样温度与惩罚项运行时重配置通道	扩展 OpenAiArgs/Anthro[span_89](start_span)[span_89](end_span)picArgs 支持运行时原子修改

Crate 名称	现有底层核心结构与职责	演化后核心新增构件	关键数据类型与 Trait 扩展规范
server	Axum 路由、JWT 认证、AppState 组装器	状态可视化端点、后台常驻心跳协程生命周期托管	/api/mind/status SSE 接口、HeartbeatHandle
thought-core	二进制入口、静态读取配置与服务初始化	自律心跳、动力学存储与显式会话的一体化引导编排	装配 HeartbeatEngine 并注入 LiveSession

核心数据结构与执行引擎代码规范

1. 认知动力学图谱结构定义（**symbol::dynamic**）

在 symbol crate 中，对原有的纯文本事实模型进行物理扩展，实现连续变量与图拓扑的紧凑内存表达：

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum DynamicNodeType {
    Concept,
    State,
    Action,
    HomeostaticDrive,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct DynamicNode {
    pub id: String,
    pub node_type: DynamicNodeType,
    pub content: String,
    pub embedding: Option<Vec<f32>>,

    // 连续动力学状态标量 (0.0 ..= 1.0)
    pub activation: f32,
    pub uncertainty: f32,
    pub tension: f32,
    pub last_tick: u64,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum EdgeKind {
    Associative {
        flux_weight: f32,
    },
    CausalTransition {
        probability: f32,
        confidence: f32,
        action_name: String,
```

```
    },
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct DynamicEdge {
    pub id: String,
    pub source: String,
    pub target: String,
    pub kind: EdgeKind,
    pub last_traversed_tick: u64,
}
```

2. 自律心跳引擎与注意门控（**agent::orchestration::heartbeat**）

在 agent crate 内部引入脱离外部 HTTP 请求的常驻驱动机制，驱动潜意识扩散并在相变时发起认知中断：

```
use std::sync::Arc;
use std::time::Duration;
use tokio::sync::broadcast;
use crate::orchestration::session::{Input, SessionHandle};

pub struct HeartbeatConfig {
    pub tick_interval: Duration,
    pub firing_threshold: f32,
    pub damping_factor: f32,
}

pub struct HeartbeatEngine {
    config: Heartbe[span_91](start_span)[span_91](end_span)atConfig,
    session: SessionHandle,
    graph: Arc<symbol::DynamicalGraphStore>,
    current_tick: u64,
}

impl HeartbeatEngine {
    pub fn new(
        config: HeartbeatConfig,
        session: SessionHandle,
        graph: Arc<symbol::DynamicalGraphStore>,
    ) -> Self {
        Self {
            config,
            session,
            graph,
            current_tick: 0,
        }
    }
```

```
    pub async fn run_loop(mut self, mut shutdown_rx: broadcast::Receiver<()>) {
        let mut interval = tokio::time::interval(self.config.tick_interval);
        loop {
            tokio::select! {
                _ = interval.tick() => {
                    self.current_tick += 1;
                    self.step_dynamics().await;
                }
                _ = shutdown_rx.recv() => {
                    tracing::info!("Heartbeat autonomic loop terminated cleanly.");
                    break;
                }
            }
        }
    }

    async fn step_dynamics(&mut self) {
        // 1. 执行快系统符号能量扩散与阻尼耗散
        let global_excitation = self.graph.propagate_activation(
            self.config.damping_factor,
            self.current_tick,
        ).await;

        // 2. 判定相变临界阈值
        if global_excitation >= self.config.firing_threshold {
            tracing::info!(excitation = global_excitation, "Cognitive ignition: Threshold exceeded");

            // 3. 提取最大张力连通子图并坍缩为语义因果断言
            if let Some(subgraph_context) = self.graph.collapse_tension_focus().await {
                // 4. 将认知中断注入 L1 主控队列，拉起慢系统思考
                let interrupt_event = Input::CognitiveInterrupt {
                    tick: self.current_tick,
                    context: Box::new(subgraph_context),
                };
                self.session.send(interrupt_event);
            }
        }
    }
}
```

3. 动态并发安全工具注册表（**plugin::dynamic_registry**）

对 plugin crate 中的单体注册表进行改造，以读写锁封装底层工具池，实现运行时的工具热挂载：

```
use std::collections::BTreeMap;
use std::sync::Arc;
use tokio::sync::RwLock;
```

```
use crate::tool::Tool;
use model::Function;

#[derive(Clone, Default)]
pub struct DynamicRegistry {
    tools: Arc<RwLock<BTreeMap<String, Arc<dyn Tool>>>>,
}

impl DynamicRegistry {
    pub fn new() -> Self {
        Self::default()
    }

    pub async fn register(&self, tool: Arc<dyn Tool>) {
        let mut write_guard = self.tools.write().await;
        write_guard.insert(tool.name().to_string(), tool);
    }

    pub async fn remove(&self, name: &str) -> bool {
        let mut write_guard = self.tools.write().await;
        write_guard.remove(name).is_some()
    }

    pub async fn definitions(&self) -> Vec<Function> {
        let read_guard = self.tools.read().await;
        read_guard.values().map(|t| t.definition()).collect()
    }

    pub async fn call(&self, name: &str, arguments: &str) -> Option<String> {
        let tool_instance = {
            let read_guard = self.tools.read().await;
            read_guard.get(name).cloned()
        };
        match tool_instance {
            Some(tool) => Some(tool.call(arguments).await),
            None => None,
        }
    }
}
```

运行时认知主状态机迁移矩阵

系统的主控制流彻底脱离单一的命令等待循环，按照确定性的认知状态机发生状态转移：

当前认知主状态	激发条件与边界判定	目标转移状态	伴随系统动作与状态副作用
Subconscious_Driftin	\mathcal{E}(t) <	Subconscious_Drifting	运行稀疏图扩散，维护

当前认知主状态	激发条件与边界判定	目标转移状态	伴随系统动作与状态副作用
g	\Theta_{\text{fire}} 且无外部中断		内稳态衰减，零外部模型调用
Subconscious_Drifting	综合兴奋度 \mathcal{E}(t) \ge \Theta_{\text{fire}}	Attention_Collapse	截断扩散循环，提取高能连通子图，计算张力梯度焦点
Attention_Collapse	子图完成结构化因果序列化	Conscious_Deliberation	封装为 Input::CognitiveInterrupt，唤醒 L1 主控推理
Conscious_Deliberation	L1 主控完成战略推理与任务下发	Hierarchical_Execution	派生 L2/L3 Worker，分配工具子集，进入局部 ReAct 循环
Hierarchical_Execution	工具调用产生非预期错误/高惊奇度	Hypothesis_Refinement	捕获反常输出，启动代码合成或因果图谱裂变更新
Hierarchical_Execution	任务成功完成并生成解决方案	Skill_Crystallization	抽取无状态代码，运行沙箱自检并注册至 DynamicRegistry
Skill_Crystallization	认知差量 \Delta[span_117](start_span)[span_117](end_span)[span_121](start_span)[span_121](end_span) C 写穿存储层	Subconscious_Drifting	释放相关节点张力，重置全局兴奋度，退回隐式动力学阶段

结论

本报告为拥有自我演化能力的非具身仿真机器人构建了完整的认知科学与系统工程蓝图。方案以全局工作空间理论（GWT）、双加工心智模型（Dual-Process Theory）与主动推理原理（Active Inference）为核心支柱，系统性消解了传统智能体对预设角色提示词（Prompt Role-Playing）与被动输入响应状态机的根本依赖。

系统实现演进的关键突破在于以下三项维度的深度闭环：其一，将 symbol crate 升级为具备能量耗散、张力积聚与因果概率更新的认知动力学图谱（CDG），确立了能够在零 Token 消耗下持续运转的隐式快系统（System 1），使系统具备了跨越时钟周期的连续存在感与自主内在动机；其二，通过全局兴奋度相变点火判决与最大子图结构化坍缩，实现了对慢系统（LLM）调用的经济性门控，并通过强类型认知差量（\Delta C）协议将局部执行噪声完全隔离于外周，彻底解决了长周期自治交互中的上下文爆炸与同质化停滞难题；其三，依托测试驱动的代码合成与动态并发安全注册表（DynamicRegistry），赋予智能体将探索世界所获经验固化为可执行程序化资产的能力，完成了从“依赖瞬时提示词推理”向“自主演进工具与世界模型”的根本性范式跃升。

基于 Rust 语言优秀的内存安全保障、轻量级异步任务调度与底层系统控制能力，该架构在现有 workspace 之上构建了高内聚、低耦合的可落地技术栈，为迈向真正意义上的自治自主演化心智系统提供了严密而清晰的工程实现路径 。

引用的文献

1. A Cognitive Architecture Based on Global Workspace Theory - arXiv, https://arxiv.org/pdf/2604.08206 2. "Theater of Mind" for LLMs: A Cognitive Architecture Based

on, https://tldr.takara.ai/p/2604.08206 3. The Architecture of Autonomy: What Would It Take to Build an Agent, https://roybatty.ca/posts/architecture-of-autonomy/ 4. A Brief account The LIDA Model and its C, https://ccrg.cs.memphis.edu/tutorial/mindAccordingToLIDA/Brief-Account.pdf 5. Principled Adaptive Routing for LLM Reasoning via Active Inference, https://arxiv.org/html/2602.23681v1 6. Active Inference for Self-Organizing Multi-LLM Systems - arXiv, https://arxiv.org/html/2412.10425v2 7. A Control-Theoretic Formulation of Global Workspace Theory - arXiv, https://arxiv.org/abs/2608.15926 8. (PDF) Cognitive Architectures for Language Agents - ResearchGate, https://www.researchgate.net/publication/373715148_Cognitive_Architectures_for_Language_Agents 9. (PDF) LIDA: A computational model of global workspace theory and, https://www.researchgate.net/publication/228621713_LIDA_A_computational_model_of_global_workspace_theory_and_developmental_learning 10. LIDA (cognitive architecture) - Wikipedia, https://en.wikipedia.org/wiki/LIDA_(cognitive_architecture) 11. LIDA Cognitive Architecture Overview | PDF - Scribd, https://www.scribd.com/document/824517581/LIDA-cognitive-architecture 12. Quick Start Tutorial - akh-medu, https://akh-medu.dev/getting-started/quickstart.html 13. Active Inference as Context Acquisition for AI Agents - arXiv, https://arxiv.org/html/2608.19202v1 14. CoALA: Awesome Language Agents - GitHub, https://github.com/ysymyth/awesome-language-agents 15. Self-Evolving Agents: A Developer's Guide - DEV Community, https://dev.to/chen115y/self-evolving-agents-a-developers-guide-40e7 16. Triggers and Diagnostics for LLM-Based Interpretability Failures in, https://arxiv.org/html/2609.23215v1 17. Self-Evolving Agents: Open-Source Projects Redefining AI in 2026, https://evoailabs.medium.com/self-evolving-agents-open-source-projects-redefining-ai-in-2026-be2c60513e97 18. VOYAGER: The First Lifelong Learning Agent in Minecraft - Skywork, https://skywork.ai/slide/en/voyager-lifelong-learning-minecraft-2069429769464324096 19. Any successful story of active inference (free energy principle)?, https://www.reddit.com/r/reinforcementlearning/comments/1fbu536/any_successful_story_of_active_inference_free/